



河北建筑工程学院

HeBei University Of Architecture

# 学士学位毕业设计

华电长青风电场测风故障检测与诊断

姓 名 侯有朔

学 号 20213090127

院 系 信息工程学院

专 业 计算机科学与技术

班 级 计 211

指导教师 董颢霞

2025 年 05 月 30 日

## 毕业设计原创性声明

本人所提交的毕业设计 \_\_\_\_\_，是在导师的指导下，独立进行研究工作所取得的原创性成果。除文中已经注明引用的内容外，本毕业设计不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中标明。

本声明的法律后果由本人承担。

毕业设计作者（签名）：

年 月 日

指导教师确认（签名）：

年 月 日

## 毕业设计版权使用授权书

本毕业设计作者完全了解河北建筑工程学院有权保留并向国家有关部门或机构送交毕业设计的复印件和磁盘，允许毕业设计被查阅和借阅。本人授权河北建筑工程学院可以将毕业设计的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或其它复制手段保存、汇编毕业设计。

保密的毕业设计在\_\_\_\_\_年解密后适用本授权书。

毕业设计作者（签名）：

年 月 日

指导教师（签名）：

年 月 日

## 摘 要

随着清洁能源的兴起和大力发展，以风电为代表的清洁能源迅猛增长，对于风电场而言，如何保持风电场稳定高效的运行是首要解决的事情。而测风仪作为风场的传感器，是预测风机功率和优化控制、保证安全和运行的重要设备。

华电长青风电场位于复杂地形高风电区域，由于复杂地形和风速导致测风系统故障率极高，而负责测风数据采集的传感器，它的微小偏差将影响风向、风速等测风数据的真值，并且恶劣天气、设备磨损等因素产生的噪声也会影响测试数据，为后续测试分析、决策判断等埋下巨大隐患。

而现有的故障检测手段难以应对这些困难，因为它主要判断阈值是否异常的极端情况，对于传感器缓慢漂移这类渐变的故障，存在检测之后，报警延迟的问题。人工巡检受人员、时间和检测人员水平的限制，使得巡检效率极其低下，且容易出现误报的情况。针对多故障耦合的复杂情况就更加难以判断故障原因，无法对故障进行有效的维修。

为此，本研究使用 Python 语言、SQLite3 数据库、ARIMA 模型搭建一套智能故障检测诊断系统结构，对数据采集、故障检测、故障诊断、故障显示等进行了实现。提高测风系统稳定性，减少发电损失，减少运维支出。

**关键词：**华青风电场；测风系统；故障分析；故障诊断

## ABSTRACT

With the rise and vigorous development of clean energy, represented by wind power, clean energy is growing rapidly. For wind farms, how to maintain the stable and efficient operation of wind farms is the primary solution. As a sensor of the wind field, the anemometer is an important equipment for predicting wind turbine power, optimizing control, ensuring safety and operation.

Huadian Changqing Wind Farm is located in a complex terrain and high wind power area. Due to the complex terrain and wind speed, the failure rate of the wind measurement system is extremely high. The sensors responsible for collecting wind measurement data have small deviations that will affect the true values of wind direction, wind speed, and other wind measurement data. In addition, noise generated by adverse weather, equipment wear, and other factors will also affect test data, laying a huge hidden danger for subsequent testing analysis, decision-making, and judgment.

However, existing fault detection methods are difficult to cope with these difficulties because they mainly judge whether the threshold is abnormal in extreme cases. For gradual faults such as slow sensor drift, there will be a problem of delayed alarm after detection. Manual inspection is limited by personnel, time, and the level of inspection personnel, which makes the inspection efficiency extremely low and prone to false alarms. It is even more difficult to determine the cause of faults in complex situations where multiple faults are coupled, making it impossible to effectively repair the faults.

Therefore, this study constructed an intelligent fault detection and diagnosis system framework based on Python language, SQLite3 database, and ARIMA model, which realizes data collection, real-time detection, fault analysis, and visual display functions. Improve the reliability of the wind measurement system, reduce power generation efficiency losses, and lower operation and maintenance costs.

**Key words:**Huaqing Wind Farm; Wind Measurement System; Fault Analysis; Fault Diagnosis

# 目 录

|                             |   |
|-----------------------------|---|
| 第 1 章 绪论 .....              | 1 |
| 1.1 研究背景 .....              | 1 |
| 1.2 国内外研究现状 .....           | 1 |
| 1.2.1 传统故障检测方法 .....        | 1 |
| 1.2.2 智能算法故障检测方法 .....      | 2 |
| 1.2.3 现有技术的局限性 .....        | 2 |
| 1.3 研究目的 .....              | 3 |
| 1.4 研究意义 .....              | 4 |
| 1.5 研究方法 .....              | 4 |
| 1.5.1 文献研究法 .....           | 4 |
| 1.5.2 数据分析法 .....           | 4 |
| 1.5.3 案例分析法 .....           | 4 |
| 第 2 章 相关技术分析 .....          | 5 |
| 2.1 Python 语言分析 .....       | 5 |
| 2.1.1 Python 语言介绍 .....     | 5 |
| 2.1.2 Python 语言特点 .....     | 5 |
| 2.2 SQLite3 数据库分析 .....     | 5 |
| 2.3 ARIMA 模型介绍 .....        | 6 |
| 2.3.1 ARIMA 模型原理 .....      | 6 |
| 2.3.2 ARIMA 模型构建步骤 .....    | 6 |
| 2.3.3 ARIMA 模型优点 .....      | 6 |
| 2.4 Streamlit 页面设计介绍 .....  | 6 |
| 2.4.1 Streamlit 特点 .....    | 6 |
| 2.4.2 Streamlit 的工作原理 ..... | 7 |
| 2.5 测风系统设备分析 .....          | 7 |
| 2.5.1 测风系统的结构 .....         | 7 |
| 2.5.2 测风系统的典型故障类型 .....     | 7 |

|                              |           |
|------------------------------|-----------|
| <b>第 3 章 系统框架设计与分析 .....</b> | <b>9</b>  |
| 3.1 系统概述 .....               | 9         |
| 3.2 系统功能分析 .....             | 10        |
| 3.2.1 数据库管理功能 .....          | 10        |
| 3.2.2 用户认证与管理功能 .....        | 10        |
| 3.2.3 故障检测功能 .....           | 10        |
| 3.2.4 系统功能模块 .....           | 10        |
| 3.2.5 系统状态管理 .....           | 11        |
| 3.3 系统的逻辑需求分析 .....          | 11        |
| 3.4 系统资源分析 .....             | 11        |
| 3.5 系统可行性分析 .....            | 12        |
| 3.5.1 技术可行性分析 .....          | 12        |
| 3.5.2 经济可行性分析 .....          | 12        |
| 3.5.3 社会可行性分析 .....          | 12        |
| <b>第 4 章 系统功能的实现 .....</b>   | <b>14</b> |
| 4.1 数据库设计 .....              | 14        |
| 4.2 用户登录与注册 .....            | 16        |
| 4.2.1 用户的登录 .....            | 16        |
| 4.2.2 用户的注册 .....            | 17        |
| 4.3 故障检测 .....               | 18        |
| 4.4 系统仪表盘 .....              | 19        |
| 4.4.1 综合视图 .....             | 19        |
| 4.4.2 最新故障记录 .....           | 20        |
| 4.4.3 风速和风向趋势 .....          | 21        |
| 4.5 故障数据 .....               | 21        |
| 4.5.1 故障数据查询 .....           | 21        |
| 4.5.2 故障统计 .....             | 23        |
| 4.5.3 故障分布 .....             | 23        |
| 4.6 数据分析 .....               | 24        |
| 4.7 故障分析 .....               | 25        |
| 4.7.1 故障统计与分布 .....          | 25        |
| 4.7.2 关联与预测 .....            | 26        |
| 4.8 参数设置 .....               | 27        |
| 4.9 用户管理 .....               | 29        |
| 4.10 系统管理 .....              | 30        |

|                          |           |
|--------------------------|-----------|
| 4.11 数据库管理 .....         | 31        |
| <b>第 5 章 系统测试 .....</b>  | <b>33</b> |
| 5.1 系统测试的目的 .....        | 33        |
| 5.2 功能性测试 .....          | 33        |
| 5.3 非功能性测试 .....         | 34        |
| 5.3.1 安全性测试 .....        | 34        |
| 5.3.2 可用性测试 .....        | 34        |
| <b>第 6 章 结论与展望 .....</b> | <b>35</b> |
| 6.1 结论 .....             | 35        |
| 6.2 未来展望 .....           | 35        |
| <b>参考文献 .....</b>        | <b>36</b> |
| <b>致谢 .....</b>          | <b>37</b> |

## 第 1 章 绪论

### 1.1 研究背景

随着全球对清洁能源需求的增加，风电行业迅速发展，风电场的高效稳定运营成为行业的关键。在这个背景下，风电场测风系统的准确性变得尤为重要。

从发电效率上看，精准的风速、风向测量数据是风机实现高效发电的基础。在瞬息万变的风场环境中，风机需要依靠测风系统最大程度地捕获风能将其转化为电能，并实时反馈风能相关信息。而灵活调整叶片角度、转速等关键参数，是确保在不同风速条件下都能最大程度地捕获风能将其转化为电能关键要素。当测风系统检测到某区内风速稳定处于风机最佳工作的风速范围内时，风机就会立马做出相应反应避免错失发电良机。而且风电场的管理人员将长期收集的测风数据通过仔细分析基本就可以掌握当地风能资源的在不同季节和不同时段的变化规律合理安排运维，进一步提升发电效率。

在安全运行方面，测风系统充当着“预警哨兵”的作用，在极端天气来临之前提前感知风速突变情况，为运维人员预留操作时间，提前采取停机、设备紧固等防护手段为风机免受暴风、暴雨、狂风等自然灾害的侵袭。例如，当测风系统感知风速即将达到风机安全停机临界值时，提前预警控制中心，运维人员远程调用停机对场内设施进行全面排查、紧固设备确保完好性，同时持续观测风场风况演化规律，也可为辨别风切变、紊流等潜在风象，为风电场选址、布防提供参考。

而华电长青风电场属复杂地形风场，集山地、丘陵、平原地形于一体，周边环境较复杂。但作为清洁能源主力，测风装置却总是出现问题。测风装置由测风塔和测风数据采集、传输装置构成，测风塔上布满各种精确度很高的传感器负责各高度天气数据采集。由于地形复杂，风向多变，传感器易受干扰，数据失真，数据漂移问题频繁出现，山区信号屏蔽、电磁干扰等导致传输异常、丢失、延时，设备老化、天气情况影响导致噪声大、数据失真，不利于风电场发电、控制及安全，也促使对该测风装置的故障检测、诊断研究。

### 1.2 国内外研究现状

#### 1.2.1 传统故障检测方法

1. 阈值判断法：确定风速、风向等的正常区间阈值，如果测量值超过或低于阈值，则可能发生了故障，例如，风速仪测量风速远高于当地历史风速或风向改变，则可初步判断发生了故障。此方法简单、方便，但需要确定阈值，容易受到环境的影响，误报率高。



2. 对比分析法：对于多台测风装置的系统可以将各个装置的测量结果进行对比；如果不同装置测量的风速、风向等的测量值在同一时刻下差异过大，超出了允许的误差范围，则说明可能有一台或者是几台都出现了故障，但是这种对比分析并不能将全部装置同时故障或者是故障情况一样的情况检测出来，对于设备安装的位置和条件要求要接近。

3. 人工巡检法：定期对测风系统的硬件设备进行检查，例如对风速仪、风向标、传感器、数据传输线路等查看是否有损坏、松动、腐蚀等物理损坏现象；硬件工作状态是否正常有无异响、振动等现象。人工巡检能够及时发现比较明显的硬件故障，但是不够及时无法实时检测，对一些隐性的硬件故障很难发现。

### 1.2.2 智能算法故障检测方法

#### 1. 基于机器学习的方法

(1) 支持向量机（SVM）：利用测风系统的历史数据，对历史数据进行特征向量的提取，用 SVM 建立风速、风向、温度、湿度参数等特征向量的故障分类模型，训练模型识别状态能力，包括正常运行状态及故障时的各种状态。SVM 在小样本上对分类效果较好，能够对一些复杂故障进行分类。

(2) 随机森林（RF）：利用多个决策树对测风数据进行分类预测。可计算高维数据，对数据的异常、噪声有一定的抗性。通过随机森林对大量的历史数据进行分析，能够学习到测风数据中各参数之间存在的复杂关系，正确判断测风系统工作状态、发现潜在故障。

(3) 人工神经网络（ANN）：模拟人类大脑的生物神经网络的结构和功能，具备非线性映射性强、学习功能性强等优点。将系统测风的所有参数作为输入，对神经网络进行训练，预测测风正常或故障状态。例如，多层感知机（MLP）可以将风速、风向时间变化规律和测风环境等其它参数进行学习，从而发现不满足正常规律的情况。

#### 2. 基于深度学习的方法

(1) 卷积神经网络（CNN）：对于测风系统图像(测风设备监看录像、传感器采集图像等)或序列数据较为方便，可以从数据中自动提取特征，通过卷积层、池化层等对图像数据进行降维并提取特征，通过全连接层进行分类并得出设备故障情况。以一个叶片图像为例，通过对正常和异常叶片特征进行学习，CNN 检测出叶片的损坏变形等故障。

(2) 循环神经网络（Recurrent Neural Network,RNN）：用于具有时间序列特征的测风数据。RNN 可以学习数据中的长期序列依赖关系，例如：LSTM 可以学习风速的时间变化趋势、不同时间序列风速的关联性，预测未来一段时间内的风速，并和实际风速进行比较，判定是否存在异常，是否存在故障。

### 1.2.3 现有技术的局限性

#### 1. 环境适应性方面

(1) 极端天气影响：对于极端天气影响如暴雨、雪、大风、沙尘等，传统的测风仪易

被损坏导致测量数据不准确，甚至无法使用。智能算法在一定程度上可以通过数据处理以及模型学习克服部分环境干扰的影响，但是对于极端天气如冰雪、沙尘堵塞等也会导致数据下降，从而导致故障检测不准确。

(2) 复杂环境干扰：测风系统所处环境复杂，例如：工业环境电磁干扰；山地复杂地形气流混乱等，都会干扰测值，传统方法无法判断故障，智能算法需要海量的数据训练模型来适应复杂环境，否则会出现误判漏判现象。

## 2. 实时性方面

(1) 数据处理速度限制：智能算法通常需要处理大量测风数据，包括历史数据和实时数据。当数据量较大时，采集、传输、处理等可能会产生一定的时差，故障检测不能及时上报，例如，深度学习算法在模型训练和推理时计算量大，需要强大的算力支持，否则无法进行实时故障检测。

(2) 系统响应时间长：对于一些智能算法模型复杂的智能检测系统，从检测到异常、发出故障报警、到执行措施之间，可能会经过很多个步骤，响应会相对比较长。而测风系统中的有些故障，需要及时处理，不然可能造成更大的影响，比如：风速仪发生故障导致风力发电机产生控制异常，长时间的延迟可能导致风机处于危险的运行状态下。

## 3. 多故障耦合问题方面

(1) 故障特征耦合：当测风系统存在多个故障，可能故障特征混合在一起，使得故障的诊断无法判断到底是哪种故障。智能算法能够学习复杂特征进行故障诊断，但是对于一些罕见多故障耦合的情况，智能算法可能在训练数据中难以学习到对应的特征进行故障诊断。

(2) 多故障耦合诊断更加困难：多故障同时发生时，故障之间可能存在关联关系，导致故障传播路径和趋势更加难以判断，故障不仅难以检测，定位和排除故障也更加困难，如风仪、风向标同时发生故障将严重影响风电场的风向风速判断，但这是两台设备独立发生的故障还是关联性故障则需更多的信息才能确定。

## 1.3 研究目的

1. 提高测风系统可靠性和稳定性：华青风电场测风系统故障率高，通过分析测风系统故障的类型、原因，有针对性地采取措施，对潜在的故障问题做到早发现、早解决，降低故障率，如完善设备维护，改进选型安装等，保障测风系统不间断运行，并提供准确测风数据。

2. 保障风电场高效稳定运行：测风数据的准确对风电场至关重要，测风系统故障会造成风力估计不准，风机工作状态不良等故障，影响到发电量和经济效益。本研究通过排查保障了风电场准确数据规划、运行与管理，让风机工作得更好，提高了风电场利用风能效率，提高风电场发电量，保障风电场稳定高效运行。

## 1.4 研究意义

华青风电场测风系统故障检测和诊断系统的研究，对提高风电场工作效率，降低运行维护成本，保证供电，确保供电质量具有十分重要的意义。

1. 提高风电场经济收益：有效的对故障进行检测、诊断，能及时对测风系统故障进行诊断、排除，确保测风数据的精确性。在精确数据的基础上，风电场优化风机布局、合理的制定运行参数，提高风能利用率提高发电量，例如避免风速误判，使风机工作在最佳状态，提高发电量，提高风电场经济收益。

2. 降低风电场运维成本：系统可以实时监视，智能诊断故障，及时确定故障点，及时分析故障原因。运维人员可以及时准备维修工具和维修零件，不要盲修，也不要乱修，降低维修成本，节省维修时间及人力物力。做好预想性维保，提前做好设备维修的部署工作，避免突发故障，减少设备损坏，降低设备维修成本，提升风电场运维管理效率。

## 1.5 研究方法

### 1.5.1 文献研究法

系统收集国内外测风系统原理、测风系统故障类型、测风系统故障诊断等研究现状、测风激光雷达控制系统研制、风场观测等技术资料，了解测风技术特点、系统故障产生机理、借鉴故障诊断方法，提供基础理论和技术借鉴，避免重复研究，少走弯路，确保研究的科学性、先进性。

### 1.5.2 数据分析法

收集华青风电场测风系统数据信息，如：采集测风系统测风数据，测风系统长时间运行数据。对数据进行统计学特征分析，寻找数据关系，如：计算数据平均值、标准差等统计数据判断是否存在异常。利用数据挖掘算法，如：通过关联规则找到故障与参数、环境之间的关联度，作为故障故障诊断数据支持，让诊断更加接近客观和真实。

### 1.5.3 案例分析法

仔细分析华青风电场发生的故障历史记录，记录下故障发生的时间，故障发生现象、发生经过、结果，通过对典型故障实例的分析总结，总结故障发生规律，故障的诊断处理过程中的经验教训，找出故障检测诊断中的难点、重点，改进系统设计、诊断流程，使系统更加有利于解决实际生产运行中遇到的各种故障。

## 第 2 章 相关技术分析

### 2.1 Python 语言分析

#### 2.1.1 Python 语言介绍

Python 是上世纪 90 年代初由荷兰国家数学和计算机科学研究机构的吉多范罗苏姆设计的，不仅拥有列表、字典等内置的有效数据结构，而且还提供了直观清晰的面向对象编程，使用简单的语句就能编写出复杂的软件开发模型。被广泛地应用于 Web 开发、科学计算、自动维护等一系列大型的开发项目中。

其解释器能够提供 C、C++ 等语言的扩充器功能，可以随意开发数据类型、算法规则等；并且，Python 源源不断的标准库提供了包括网络通信、文件处理、图形界面等，提供源码、二进制文件等多种适合各种平台的开发源码及代码，为开发者提供各种底层支持。

#### 2.1.2 Python 语言特点

1. 简单易懂：Python 代码就像流利的英语一样规范清晰，即使是初学者也能明白代码的含义，让读者只需专注于要解决的问题，而不是浪费时间去搞清楚语法规则。

2. 跨平台性：它可以在 Windows、Linux、Mac OS 等多个平台上运行，而且可以在各个平台上取得一致的效果。用户编写的 Python 代码不需要进行修改或者进行少量的修改就可以在所有平台上使用。

3. 丰富的标准库和第三方库：Python 本身有丰富的标准库，包括文件操作、网络操作、数据操作等，可以实现快速操作，同时，Python 社区还提供了很多第三方库，比如 NumPy 用于科学计算，Pandas 用于数据分析，Django 用于 Web 开发等。

### 2.2 SQLite3 数据库分析

SQLite3 是一个轻型的嵌入式数据库管理系统，以文件的形式存在，没有独立的服务器进程，在许多领域都能发现其身影。它的特点如下：

1. 轻量级：SQLite3 占用资源少，对于资源受限设备对内存空间和磁盘空间的利用率都较高。

2. 零配置：在使用上，不需要做额外的配置，只要有 SQLite 库、对应的文件后就可以直接使用。

3. 自给自足：数据库以单个文件形式存在，便于携带和备份。

4. 事务支持：支持事务操作，确保数据的一致性和完整性。

## 2.3 ARIMA 模型介绍

### 2.3.1 ARIMA 模型原理

ARIMA 模型通过对时间序列数据进行差分处理，将非平稳序列转化为平稳序列，然后建立自回归（AR）模型和滑动平均（MA）模型来描述平稳序列的变化规律。

自回归（AR）部分：考虑当前值与过去值之间的线性关系，例如  $AR(p)$  模型表示当前值是过去  $p$  个观测值的线性组合再加上一个误差项。

滑动平均（MA）部分：关注误差项的相关性， $MA(q)$  模型表示当前的误差项是过去  $q$  个误差项的线性组合。

积分（I）部分：用于处理非平稳时间序列，通过对序列进行  $d$  阶差分，使其变为平稳序列， $d$  表示差分的阶数。

### 2.3.2 ARIMA 模型构建步骤

1. 准备数据：从原始的数据中提取时间序列，并将其设为索引方便后续分析。
2. 模型构造与训练：采用 ARIMA 类的构造方法模型实体，改变 AR、MA、I 参数值，对比故障结果选择最佳值。
3. 预测：使用训练好的 ARIMA 模型对未来数据进行预测。

### 2.3.3 ARIMA 模型优点

ARIMA 模型能够较好地处理线性时间序列数据，对短期预测有较高的准确性，并且具有坚实的理论基础和成熟的估计与检验方法。

## 2.4 Streamlit 页面设计介绍

Streamlit 是一款快速构建数据 App 的开源 Python 库，该库于 2018 年发布，作者是 Adrien Treuille、Amar Bakshi、Amaraki Teixeira。

### 2.4.1 Streamlit 特点

1. 简洁性：Streamlit 简化了 Web 应用开发过程，使用纯 Python 代码即可开发交互式应用，无需学习 Web 前端应用相关知识，降低了 Web 应用开发门槛。
2. 快速迭代：采用“一次编写，立即运行”的编写方式，修改代码后立即在浏览器上看到修改后的运行效果。极大的提升了开发效率。
3. 丰富的组件库：提供包括按钮、滑块、下拉框、文本输入框在内的组件来搭建交互界面，实现交互式功能。

4. 可视化集成：数据可视化库集成简单，比如：Matplotlib, Plotly, Altair 等。可视化集成可以简单展示分析数据结果。

### 2.4.2 Streamlit 的工作原理

1. 执行脚本：Streamlit 应用本质上是一个 Python 脚本，当用户在浏览器中访问应用时，Streamlit 会执行该脚本。

2. 状态管理：它会跟踪脚本中变量的状态，当用户与界面组件进行交互时，Streamlit 会重新运行脚本的部分代码，并更新界面以反映最新的状态。

3. 服务器通信：Streamlit 会在本地启动一个 Web 服务器，将 Python 脚本的输出转换为 HTML 和 JavaScript 代码，然后在浏览器中渲染。

## 2.5 测风系统设备分析

测风系统是风电场及气象领域用于实时测量风速、风向、气压等气象参数的核心设备，其数据直接影响风电场的发电量评估、风机控制策略优化、电网调度规划等关键环节。

### 2.5.1 测风系统的结构

测风系统由传感器部分、采集与数据处理部分、传输部分和电源部分构成：

1. 传感器单元：传感器单元通常由风速传感器、风向传感器、辅助传感器组成，风速传感器通常有机械式传感器及超声波传感器，机械式传感器由转轴带动编码器把输出信号输出脉冲信号；超声波式是采用时差法进行传感器的非接触测量，通过顺风向与逆风声波传播时间差进行计算风速；风向传感器由风向标和角度编码器组成，风向标用来进行风向的指向，角度编码器将传感器的传入（电信号或其他）信号转换为  $0^{\circ} \sim 360^{\circ}$ 。辅助传感器通常有温度、气压、湿度传感器等。

2. 数据采集处理单元：包括微控制器和各种传感器的信号的数据采集模块，完成 A/D 转换，温度补偿、零点漂移补偿、信号滤波、平均风速计算、风向变化区间计算等。

3. 传输单元：分为有线和无线传输两种方式，有线传输使用屏蔽双绞线或光纤进行传输，无线传输短距离内可以使用 Wifi、蓝牙，长距离可以选择 4G/5G 模块或低能耗的局域网。

4. 供电单元：通常使用风机机舱电源并且配备防雷措施防止传感器和电源受到雷击损坏。

### 2.5.2 测风系统的典型故障类型

传感器故障：主要是风速传感器和风向传感器故障，例如传感器数据跳变、数据不稳定、数据与实际风况不对应等。辅助传感器故障，例如温度传感器短路。

数据处理和传输故障：信号中断、数据失准。信号中断故障可能是由于电缆接头异常、

总线芯片故障等其他原因引起的。数据效验失准是由于设备时间混乱、传感器丢失数据参数等其他原因引起的。

涉及环境的故障：机械式传感器冻冰不能转动，超声波传感器探头覆盖有雪不能测量。雷电传感器内部的电路被击穿等。

## 第3章 系统框架设计与分析

### 3.1 系统概述

本系统是风电场测风系统故障检测和诊断系统，通过对风电场测风数据的风速与风向进行检测、分析与诊断，完成用户管理、系统管理等操作。本系统使用 Python 和 Streamlit 构建，通过构建 SQLite 数据库来实现，具体过程如图 3.1 所示。

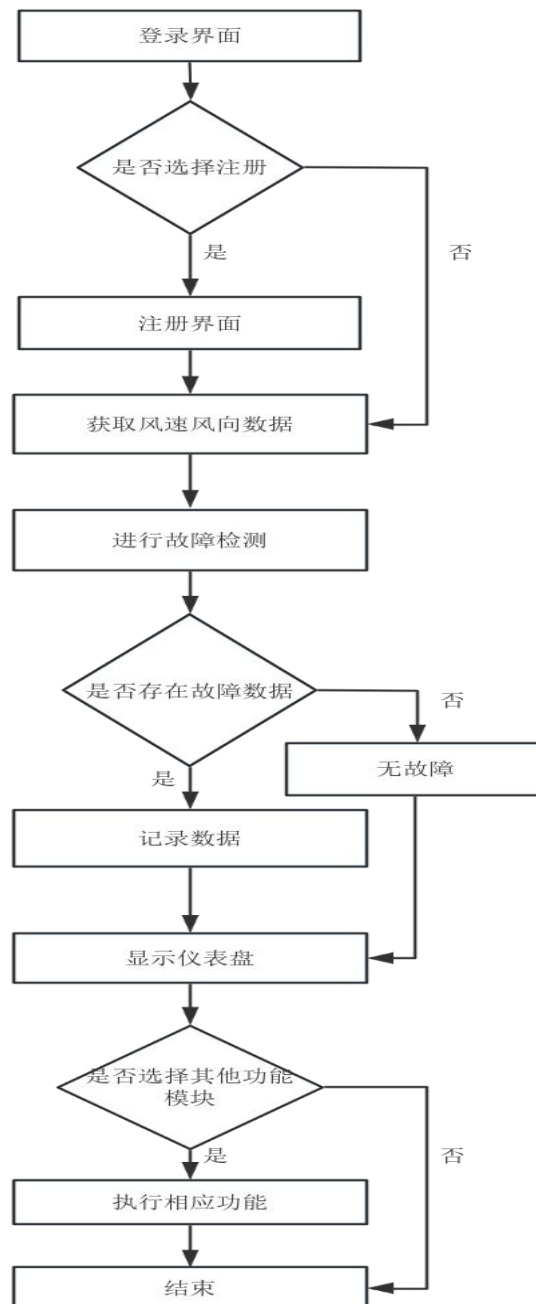


图 3.1 系统流程图



## 3.2 系统功能分析

### 3.2.1 数据库管理功能

建立与 SQLite 数据库的连接。在系统运行过程中，任何涉及到数据库操作的功能都要通过调用函数获取连接对象，从而确保能够对数据库进行读写操作。并且当系统首次运行时，系统会自动调用函数创建必要的表，如 `users` 表用于存储用户信息，为后续的数据存储和管理奠定基础。

### 3.2.2 用户认证与管理功能

建立登陆界面，用户通过输入用户名和密码进行登录。通过调用函数将用户输入的用户名与密码与数据库中存储的信息进行比对认证，认证通过后用户才能登录系统并使用相关功能，从而保证了系统的安全性。同时在登录界面还提供注册按钮，方便新用户注册。

在注册的过程中，用户需要输入用户名、密码、邮箱、电话等信息，系统会对这些信息进行格式验证，验证通过后将用户信息存储到数据库中。

### 3.2.3 故障检测功能

系统会根据预设的风速故障阈值，对风速数据进行分析。可以指定特定的风机编号，检测该风机的风速是否超出正常范围，若超出则判定为故障。风向故障检测与风速故障检测类似，根据风向故障阈值对风向数据进行检测，判断风向是否出现异常。

### 3.2.4 系统功能模块

1. 仪表盘：作为首页，应该体现出系统中实时数据，如风速、风向、故障数等，以图表和报表形式体现，让用户对风电场运行情况一目了然。

2. 系统管理：对系统进行配置和管理，如系统参数配置、系统日志管理等，保证系统的正常运作。

3. 用户管理：用户管理功能可以实现对系统中的用户信息进行查看、修改、删除，保证用户信息的准确性和安全性。

4. 故障数据管理：用于存储检测到的故障数据，提供故障数据记录查询、统计、分析的功能，方便用户了解故障数据。

5. 数据分析：对风速、风力的数据分析，找出相应的规律和趋势，为风电场的运维决策提供辅助。

6. 参数的设置：这一功能能够使用户根据自身情况设置系统的各种参数，包括故障阈值、采集数据的频率等，使系统适用于不同的场合。

7. 故障分析：分析故障数据，定位故障原因，识别故障的影响因素，提出解决方案和建议，帮助用户尽快解决故障。

8. 数据库管理：通过数据库维护、管理，备份、恢复、优化等功能，实现数据库的稳定运行。

### 3.2.5 系统状态管理

管理系统的各种状态，如用户认证状态、用户名、用户角色、注册模式等。这种状态管理方式使得系统在不同页面和功能之间能够保持状态的一致性，从而提高用户体验。

## 3.3 系统的逻辑需求分析

第一，需要创建数据库，存储相应的数据信息，并且要具备数据库连接功能，以支持系统与数据库之间的数据交互。当数据库文件不存在时，能自动初始化数据库结构，创建必要的表。同时要支持数据的插入和查询操作，为后续功能模块的开发提供基础。

第二，用户在登录界面，可以实现用户的登录，并可以通过旁边的注册按钮，跳转到注册页面，通过输入基本信息进行注册。

第三，在故障检测模块中，可以根据预设的风速风向故障阈值，对指定的风机或全部风机的数据进行分析判断是否发生故障。

第四，在各个功能模块中，都能实现对应的功能，例如：在仪表盘界面，展示关键数据和指标，如实时风速、风向、故障数量等并且采用可视化图表和报表形式呈现数据，便于用户直观了解风电场运行状况。在系统管理模块，提供系统整体设置和配置功能，如调整系统参数、管理系统日志等。只有具备相应权限的用户（如系统管理员）才能进行操作等。

第五，管理系统的各种状态。保证系统在不同页面和功能之间切换时，保持状态一致，避免出现数据丢失或错误。并且当用户登录或注销时，及时更新用户认证状态和相关信息。

## 3.4 系统资源分析

从硬件方面上看，该系统需要强大的 CPU 和足够的内存进行支撑。因为其需要对风速、风向数据进行实时处理和分析，同时还要处理用户的登录、注册请求以及数据库的读写操作。所以 CPU 需要具备一定的计算能力。内存是存储系统运行数据和程序运行数据的地方，系统需要载入数据、缓存风速风向数据、保存用户的会话信息等，所以系统需要内存来存储相应的数据维持系统的运行。

从软件角度分析，可以选择 Windows Server 系统，操作简单，对熟悉 Windows 操作的人员更容易掌握，系统需要大量的数据，需要数据库系统对其进行存储和管理，并且需要一个必要的发展工具和编程语言来对系统的数据和页面设计进行维护。

从数据资料方面来讲，系统需要大量的风速风向数据资料，数据资料的准确和及时性直接影响着系统的故障检测和诊断的准确性，因此需要对数据进行定时采集并存储。需要

对数据进行备份和恢复，以免数据丢失。

除此之外，系统需要随时接收风速风向数据，用户需要从系统接收网络信息，系统要有足够网络带宽以保证系统接收信息的速度。

### 3.5 系统可行性分析

#### 3.5.1 技术可行性分析

编程语言和框架：开发语言选择 Python，其具备丰富的库和工具，例如 Pandas 可以有效地加工与分析数据，方便阅读数据，例如风速、风向等；Matplotlib 和 Plotly 可以将数据转换为图形，让人们更好地分析数据，了解数据特征。Streamlit 框架可以节省大量的 Web 应用开发时间，快速构建交互性强、用户体验感高的前端页面。

数据库管理：使用的数据库为轻量嵌入式数据库 SQLite 数据库，该数据库是成熟稳定的数据库，并且容易嵌入系统，不需要服务器的配置，适用于小系统，而对于日后系统的发展，也容易扩展到更大型的数据库，如 MySQL。

故障检测算法：系统故障检测的功能主要是通过阈值对于风速以及风向的数据进行检测，其算法较为简单，易学易懂，也便于维护。后续如果需要进行功能开发，可以采用更为复杂的机器学习算法，例如基于深度学习异常检测算法，进行更高效的故障检测。

系统架构：架构简洁、模块相对独立、易开发、易维护。系统采用模块化的架构设计，方便实现各个功能的开发和维护，可开发或更改不同的功能，扩展、维护容易。

#### 3.5.2 经济可行性分析

第一，对系统开发所需要的成本进行分析。首先由于使用了成熟的技术和框架，开发人员无需具备过高的专业技能，降低了人力成本。而且，Streamlit 框架提高了开发效率，缩短了开发周期，进一步减少了人力投入。其次 Python、Streamlit、SQLite 等均为开源软件，无需支付软件授权费用，大大降低了软件开发的成本。最后系统对硬件资源的要求不高，对于小型风电场，普通的服务器或个人计算机即可满足需求，降低了硬件采购成本。

第二，对系统运营成本进行分析，系统运行时对硬件资源的占用较少，能源消耗相对较低，降低了长期的运营成本。并且系统的模块化设计使得维护工作更加简单和高效。当出现故障时，可快速定位和修复问题，减少了维护时间和成本。同时，系统的日志记录和监控功能有助于及时发现潜在问题，进行预防性维护。

第三，对系统的收益和效益进行分析。本系统可以及时检测和诊断故障，减少风电机组的停机时间，提高风电场的发电效率，从而增加电力销售收入。此系统也可以提前发现潜在故障，采取预防性维护措施，可降低维修成本和更换零部件的费用。

#### 3.5.3 社会可行性分析

首先随着全球对清洁能源的重视和推广，各国政府纷纷出台相关政策支持风力发电产业的发展。风电场测风系统故障检测与诊断系统有助于提高风电场的运行效率和可靠性，符合国家能源发展战略和政策导向，能够得到政策法规的支持。并且风力发电本身就是一种清洁能源，该系统的应用有助于提高风电场的发电效率，减少对传统化石能源的依赖，降低碳排放，对环境保护具有积极意义。其次此系统的开发、维护和运营需要一定的专业人才，能够创造相关的就业机会，促进当地经济发展。最后本系统的 Web 界面设计简洁直观，操作方便，用户无需具备专业的技术知识即可轻松上手，而且系统提供了丰富的功能和数据展示，能够满足用户对风电场运行状况的监测和管理需求，提高用户的工作效率和决策准确性，因此容易被用户接受。

## 第 4 章 系统功能的实现

### 4.1 数据库设计

系统需要建立多个数据表格用来存储不同数据，主要表格包括：用户表（users）、风电机组表（turbines）、风速数据表（wind\_data）、故障记录表（fault\_records）、阈值表（fault\_thresholds），他们的具体样式和存储内容如下：

表 4.1 users 表是用来存储用户的基本信息，为用户认证和管理提供数据支持。系统可以通过该表对用户登录、注册进行验证，同时也方便管理员对用户信息进行管理。

表 4.1 users 表

| 字段名               | 数据类型 | 描述     | 约束条件        |
|-------------------|------|--------|-------------|
| username          | TEXT | 用户名    | PRIMARY KEY |
| password          | TEXT | 密码     | NOT NULL    |
| role              | TEXT | 角色     | NOT NULL    |
| name              | TEXT | 姓名     | NOT NULL    |
| email             | TEXT | 邮箱     | NOT NULL    |
| phone             | TEXT | 电话     | NOT NULL    |
| last_login        | TEXT | 上次登录时间 | 可以为空        |
| registration_date | TEXT | 注册日期   | NOT NULL    |

表 4.2 turbines 表是用来存储风电机组的基本信息进行存储，方便系统对当前风电机组的型号、风机的数量进行统计，另外此表后续会与 wind\_data 表和 fault\_records 表会通过 turbine\_id 与其建立关联。

表 4.2 turbines 表

| 字段名               | 数据类型 | 描述         | 约束条件        |
|-------------------|------|------------|-------------|
| turbine_id        | TEXT | 风电机组唯一的标识符 | PRIMARY KEY |
| name              | TEXT | 名称         | NOT NULL    |
| location          | TEXT | 安装位置       | 可以为空        |
| installation_date | TEXT | 安装日期       | NOT NULL    |
| status            | TEXT | 当前状态       | 可以为空        |

表 4.3 wind\_data 表是用于存储风速风向等相关数据，记录风电机组传入的风速风向，

温度、湿度、气压、可能存在的故障等信息，并通过 FOREIGN KEY(turbine\_id) REFERENCES turbines (turbine\_id) 建立了与 turbines 表的关联，确保 turbine\_id 的值在 turbines 表中存在。

表 4.3 wind\_data 表

| 字段名            | 数据类型    | 描述                             | 约束条件                             |
|----------------|---------|--------------------------------|----------------------------------|
| id             | INTEGER | 每条记录的唯一<br>自增 ID               | PRIMARY KEY<br>AUTOINCREM<br>ENT |
| timestamp      | TEXT    | 数据的采集时间                        | NOT NULL                         |
| turbine_id     | TEXT    | 关联 turbines 表<br>中的 turbine_id | NOT NULL                         |
| wind_speed     | REAL    | 风速                             | 可以为空                             |
| wind_direction | REAL    | 风向                             | 可以为空                             |
| temperature    | REAL    | 温度                             | 可以为空                             |
| humidity       | REAL    | 湿度                             | 可以为空                             |
| air_pressure   | REAL    | 气压                             | 可以为空                             |
| fault_type     | TEXT    | 可能出现的故障<br>类型                  | 可以为空                             |

表 4.4 fault\_records 表是用于存储风电机组的故障记录。记录风电机组故障发生的时间和故障类型，并根据故障类型生成相关的故障描述和处理建议，并通过 FOREIGN KEY (turbine\_id) REFERENCES turbines (turbine\_id) 与 turbines 表建立关联，确保故障记录与对应的风电机组关联。

表 4.4 fault\_records 表

| 字段名            | 数据类型    | 描述                             | 约束条件                             |
|----------------|---------|--------------------------------|----------------------------------|
| id             | INTEGER | 每条记录的唯一<br>自增 ID               | PRIMARY KEY<br>AUTOINCREM<br>ENT |
| timestamp      | TEXT    | 故障发生的时间                        | NOT NULL                         |
| turbine_id     | TEXT    | 关联 turbines 表<br>中的 turbine_id | NOT NULL                         |
| fault_type     | TEXT    | 故障的类型                          | NOT NULL                         |
| description    | TEXT    | 故障详细描述                         | 可以为空                             |
| recommendation | TEXT    | 故障的处理建议                        | 可以为空                             |
| status         | TEXT    | 故障的处理状态                        | 默认值为“未处<br>理”                    |

表 4.5 fault\_thresholds 表是用于存储故障检测的阈值信息，记录阈值的名称和初始数值，并对阈值进行详细的描述，后续可以根据实际情况，对阈值数值进行修改，并记录最后的修改时间。

表 4.5 fault\_thresholds 表

| 字段名          | 数据类型    | 描述         | 约束条件                         |
|--------------|---------|------------|------------------------------|
| id           | INTEGER | 记录唯一的自增 ID | PRIMARY KEY<br>AUTOINCREMENT |
| name         | TEXT    | 阈值的名称      | NOT NULL                     |
| value        | REAL    | 阈值的具体数值    | NOT NULL                     |
| description  | TEXT    | 阈值的详细描述    | 可以为空                         |
| last_updated | TEXT    | 阈值的最后更新时间  | 可以为空                         |

## 4.2 用户登录与注册

### 4.2.1 用户的登录

通过 authenticate 函数实现用户的登录功能。用户输入的用户名和密码，系统会在数据库的 users 表中的寻找相应数据，并将输入的密码与表中密码进行哈希比较，如果比对通过，则更新用户最后登录时间，并将用户的认证状态和角色信息保留到会话状态中。具体界面如图 4.1 所示。



图 4.1 登录界面

核心代码如下：

```
if username in st.session_state.users:
```

```

hashed_password = hashlib.sha256(password.encode()).hexdigest()
if stored_password == hashed_password:
    st.session_state.authenticated = True
    st.session_state.username = username
    st.session_state.user_role = st.session_state.users[username]["role"]
    st.session_state.users[username]["last_login"] = datetime.datetime.now().strftime
("%Y-%m-%d %H:%M:%S")
    return True
return False

```

#### 4.2.2 用户的注册

通过 `registration_page` 函数实现用户的注册功能，用户通过输入用户名、密码、确认密码、邮箱、电话等信息后，系统会对用户名是否唯一，邮箱、电话是否格式正确进行判断，如果通过判断，则将用户的信息添加到数据库的 `users` 表中，如果未通过验证，则进行提醒，要求用户修改相关信息。具体界面如图 4.2 所示。

华电长青风电场



## 华电长青风电场测风系统故障检测与诊断系统

### 用户注册

用户名

密码

确认密码

姓名

邮箱

电话

图 4.2 注册页面

核心代码如下：

```

if st.button(“注册”):
    if not new_username or not new_password or not confirm_password or not new_n
ame or not new_email or not new_phone:

```

17



```

        st.error(“请填写所有必填信息。”)
    elif new_password != confirm_password:
        st.error(“两次输入的密码不一致。”)
# 添加新用户
hashed_password = hashlib.sha256(new_password.encode()).hexdigest()
registration_date = datetime.datetime.now().strftime(“%Y-%m-%d %H:%M:%S”)
role = “操作员”
cursor.execute(“INSERT INTO users (username, password, role, name, email, phone,
registration_date) VALUES (?, ?, ?, ?, ?, ?, ?)”,(new_username, hashed_password, role, new_name, new_email, new_phone, registration_date))

```

### 4.3 故障检测

创建 detect\_wind\_speed\_faults 函数对风速数据进行检测，找出风速为 0、信号丢失、传感器卡滞等故障情况，并生成相应的故障记录。创建 detect\_wind\_direction\_faults 函数对风向数据进行检测，找出风向为 239° 不变、风向为 0°、信号丢失、传感器卡滞等故障情况，并生成相应的故障记录。从而实现故障检测，为后面的数据统计和可视化提供基础。

核心代码如下（以风速数据为 0 为例）：

```

if turbine_id:
    data = df[df[“turbine_id”] == turbine_id].copy()
else:
    data = df.copy()
faults = []
for turbine in data[“turbine_id”].unique():
    turbine_data = data[data[“turbine_id”] == turbine]
    # 检测风速为 0 的情况
    zero_speed = turbine_data[turbine_data[“wind_speed”] == 0]
    if len(zero_speed) > 0:
        for _, row in zero_speed.iterrows():
            faults.append({“timestamp” : row[“timestamp”],
                            “turbine_id” : turbine,
                            “fault_type” : “风速传感器可能故障”,
                            “description” : “风速示值为 0m/s, 可能存在供电系统问题”,
                            “recommendation” : “检查电缆线和电源供电系统是否有问
题” })
    return pd.DataFrame(faults) if faults else pd.DataFrame(

```

```
columns=["timestamp", "turbine_id", "fault_type", "description", "recommendation"]])
```

## 4.4 系统仪表盘

### 4.4.1 综合视图

创建 `dashboard_combined` 函数用来展示系统的关键指标：风电机组总数、当前故障总数、平均风速。并使用 Plotly 绘制图表展示故障分布情况，包含故障类型分布、各风机故障数量，具体界面如图 4.3 所示。

## 系统仪表盘 - 综合视图

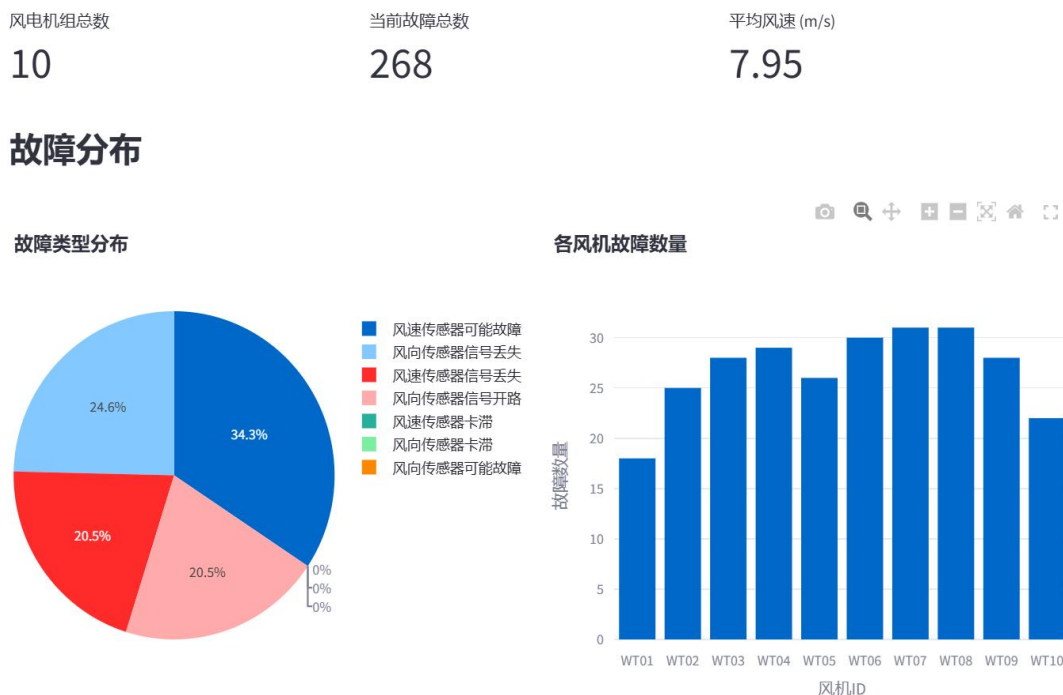


图 4.3 综合视图

核心代码如下：

```
st.metric(label=“风电机组总数”，value=“10”，delta=None)
wind_speed_faults = detect_wind_speed_faults(st.session_state.wind_data)
wind_direction_faults = detect_wind_direction_faults(st.session_state.wind_data)
total_faults = len(wind_speed_faults) + len(wind_direction_faults)
st.metric(label=“当前故障总数”，value=total_faults, delta=None)
st.metric(label=“平均风速 (m/s)”，value=f“{st.session_state.wind_data['wind_speed']}”)
```

```
mean():.2f}" , delta=None)
```

```
fault_df = pd.DataFrame({"故障类型": list(fault_data.keys()), "数量": list(fault_data.values())})
```

```
fig = px.pie(fault_df, names="故障类型", values="数量", title="故障类型分布")
st.plotly_chart(fig, use_container_width=True)
```

```
fig = px.bar(turbine_fault_df, x="风机 ID", y="故障数量", title="各风机故障数量")
```

```
st.plotly_chart(fig, use_container_width=True)
```

#### 4.4.2 最新故障记录

创建 `dashboard_latest_faults()` 函数调用 `detect_wind_speed_faults` 和 `detect_wind_direction_faults` 方法, 分别检测风速和风向的故障数据, 并两部分故障数据合并和按时间降序排序, 在表中显示最近 10 条故障记录, 具体界面如图 4.4 所示。

### 系统仪表盘 - 最新故障记录

|     | 时间                  | 风机ID | 故障类型      | 描述                    | 建议                |
|-----|---------------------|------|-----------|-----------------------|-------------------|
| 43  | 2025-04-12 07:02:01 | WT04 | 风速传感器可能故障 | 风速示值为0m/s, 可能存在供电系统问题 | 检查电缆线和电源供电系统是否有问题 |
| 21  | 2025-04-12 01:02:01 | WT02 | 风速传感器信号丢失 | 风速传感器信号丢失             | 检查传感器连接和供电        |
| 125 | 2025-04-11 19:02:01 | WT09 | 风速传感器可能故障 | 风速示值为0m/s, 可能存在供电系统问题 | 检查电缆线和电源供电系统是否有问题 |
| 133 | 2025-04-11 17:02:01 | WT09 | 风速传感器信号丢失 | 风速传感器信号丢失             | 检查传感器连接和供电        |
| 56  | 2025-04-11 15:02:01 | WT05 | 风向传感器信号丢失 | 风向传感器信号丢失             | 检查传感器连接和供电        |
| 50  | 2025-04-11 11:02:01 | WT05 | 风向传感器信号开路 | 风向示值为239°不变, 可能是信号开路  | 检查接插件和电缆          |
| 54  | 2025-04-11 07:02:01 | WT04 | 风速传感器信号丢失 | 风速传感器信号丢失             | 检查传感器连接和供电        |
| 53  | 2025-04-11 05:02:01 | WT04 | 风速传感器信号丢失 | 风速传感器信号丢失             | 检查传感器连接和供电        |
| 111 | 2025-04-11 05:02:01 | WT09 | 风向传感器信号丢失 | 风向传感器信号丢失             | 检查传感器连接和供电        |
| 18  | 2025-04-11 03:02:01 | WT02 | 风速传感器可能故障 | 风速示值为0m/s, 可能存在供电系统问题 | 检查电缆线和电源供电系统是否有问题 |

图 4.4 最新故障记录

核心代码如下:

```
wind_speed_faults = detect_wind_speed_faults(st.session_state.wind_data)
wind_direction_faults = detect_wind_direction_faults(st.session_state.wind_data)
all_faults = pd.concat([wind_speed_faults, wind_direction_faults])
all_faults["timestamp"] = pd.to_datetime(all_faults["timestamp"])
all_faults = all_faults.sort_values("timestamp", ascending=False)
st.dataframe(all_faults.head(10)[["timestamp", "turbine_id", "fault_type", "description", "recommendation"]].rename(columns={
    "timestamp": "时间",
    "turbine_id": "风机 ID",
```

```

“fault_type”：“故障类型”，
“description”：“描述”，
“recommendation”：“建议”
}), use_container_width=True)

```

#### 4.4.3 风速和风向趋势

创建 `dashboard_wind_speed_direction_trends` 函数。第一，获取风速数据，转换时间戳为日期格式，按天计算每日平均风速，使用 Plotly 绘制每日平均风速趋势图并显示。第二，获取过去 24 小时内的风速和风向数据。使用极坐标散点图展示过去 24 小时的风速和风向分布，并按风力发电机 ID 着色，调整图表布局后进行显示，具体界面如图 4.5 所示。

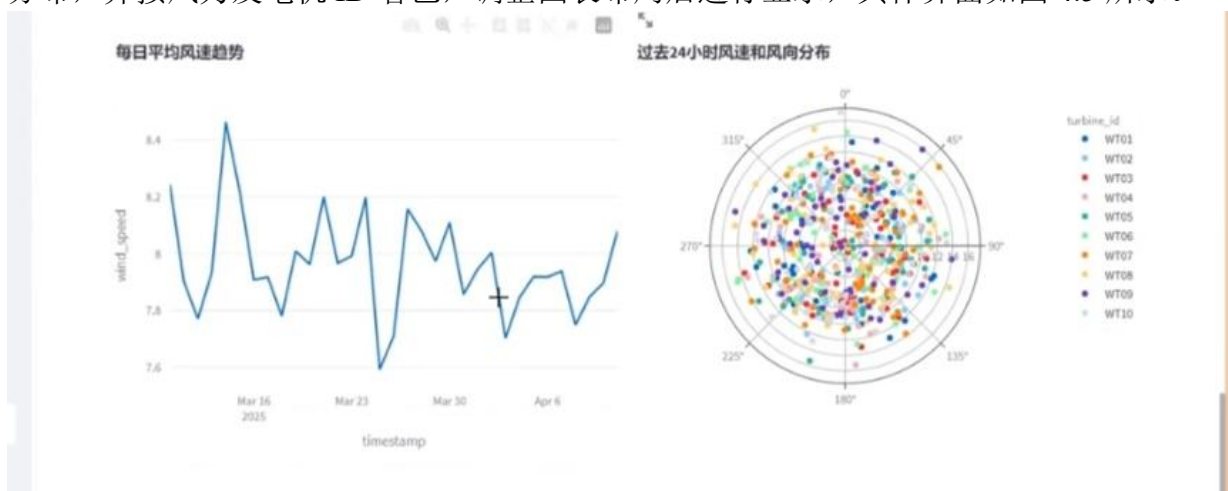


图 4.5 风速风向趋势

代码如下：

```

fig = px.line(df_avg, x="timestamp", y="wind_speed", title="每日平均风速趋势")
st.plotly_chart(fig, use_container_width=True)

fig = px.scatter_polar(df_24h, r="wind_speed", theta="wind_direction", color="turbine_id", title="过去 24 小时风速和风向分布")
fig.update_layout(polar=dict(radialaxis=dict(showticklabels=True, ticks="")))
st.plotly_chart(fig, use_container_width=True)

```

## 4.5 故障数据

### 4.5.1 故障数据查询

用户可以选择需要查询的风机、故障类型和时间范围，系统会调用 `detect_wind_speed_faults` 和 `detect_wind_direction_faults` 函数检测风速和风向传感器的故障数据。并根据用户需求对数据进行合并或筛选，并按照时间范围进行过滤，将数据显示在表格中，供用户查

看和导出。具体页面如图 4.6 所示。

故障数据查询

选择风机

故障类型

日期范围

全部

全部

2025/04/01 - 2025/04/10

查询

|    | 时间                  | 风机ID | 故障类型      | 描述                    | 建议                |
|----|---------------------|------|-----------|-----------------------|-------------------|
| 8  | 2025-04-10 15:02:01 | WT01 | 风速传感器可能故障 | 风速示值为0m/s, 可能存在供电系统问题 | 检查电缆线和电源供电系统是否有问题 |
| 9  | 2025-04-04 13:02:01 | WT01 | 风速传感器信号丢失 | 风速传感器信号丢失             | 检查传感器连接和供电        |
| 15 | 2025-04-01 15:02:01 | WT02 | 风速传感器可能故障 | 风速示值为0m/s, 可能存在供电系统问题 | 检查电缆线和电源供电系统是否有问题 |
| 16 | 2025-04-04 21:02:01 | WT02 | 风速传感器可能故障 | 风速示值为0m/s, 可能存在供电系统问题 | 检查电缆线和电源供电系统是否有问题 |
| 17 | 2025-04-08 03:02:01 | WT02 | 风速传感器可能故障 | 风速示值为0m/s, 可能存在供电系统问题 | 检查电缆线和电源供电系统是否有问题 |
| 25 | 2025-04-01 01:02:01 | WT03 | 风速传感器可能故障 | 风速示值为0m/s, 可能存在供电系统问题 | 检查电缆线和电源供电系统是否有问题 |
| 26 | 2025-04-03 01:02:01 | WT03 | 风速传感器可能故障 | 风速示值为0m/s, 可能存在供电系统问题 | 检查电缆线和电源供电系统是否有问题 |
| 27 | 2025-04-04 11:02:01 | WT03 | 风速传感器可能故障 | 风速示值为0m/s, 可能存在供电系统问题 | 检查电缆线和电源供电系统是否有问题 |
| 28 | 2025-04-09 05:02:01 | WT03 | 风速传感器可能故障 | 风速示值为0m/s, 可能存在供电系统问题 | 检查电缆线和电源供电系统是否有问题 |
| 29 | 2025-04-10 07:02:01 | WT03 | 风速传感器可能故障 | 风速示值为0m/s, 可能存在供电系统问题 | 检查电缆线和电源供电系统是否有问题 |

导出为CSV

图 4.6 故障数据查询

```
核心代码如下：
if st.button(“查询”):
    wind_speed_faults = detect_wind_speed_faults(st.session_state.wind_data,None if sele
cted_turbine == “全部” else selected_turbine)
    wind_direction_faults = detect_wind_direction_faults(st.session_state.wind_data,
None if selected_turbine == “全部” else selected_turbine)
    if fault_type == “风速传感器故障”：
        all_faults = wind_speed_faults
    elif fault_type == “风向传感器故障”：
        all_faults = wind_direction_faults
    else:
        all_faults = pd.concat([wind_speed_faults, wind_direction_faults])
    start_date = pd.Timestamp(date_range[0])
    end_date = pd.Timestamp(date_range[1]) + pd.Timedelta(days=1) - pd.Timedelta(sec
```

```
onds=1)
```

```
all_faults[ "timestamp" ] = pd.to_datetime(all_faults[ "timestamp" ])
```

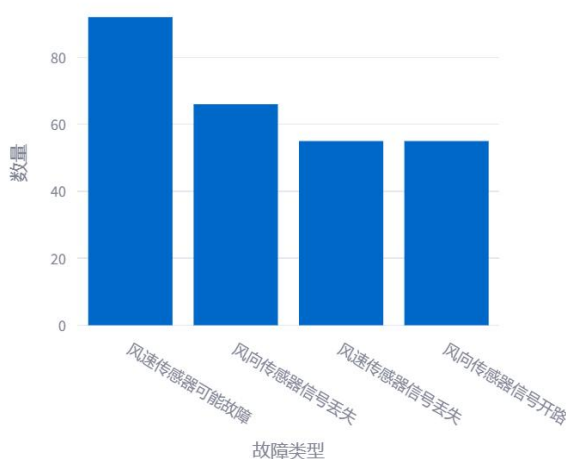
```
filtered_faults = all_faults[(all_faults[ "timestamp" ] >= start_date) & (all_faults[ "timestamp" ] <= end_date)]
```

#### 4.5.2 故障统计

调用 `detect_wind_speed_faults` 和 `detect_wind_direction_faults` 函数，分别检测风速和风向的故障数据，并将结果进行合并。根据统计的数据绘制两个图表：一个是统计各类故障的数量，并以柱状图展示。另一个则按日期统计每日故障数量，并以折线图展示。具体界面如图 4.7 所示。

##### 故障统计

故障类型统计



每日故障数量趋势



图 4.7 故障统计图

代码如下：

```
fig = px.bar(fault_counts, x= "故障类型", y= "数量", title= "故障类型统计")
```

```
st.plotly_chart(fig, use_container_width=True)
```

```
fig = px.line(daily_counts, x= "日期", y= "故障数量", title= "每日故障数量趋势")
```

```
st.plotly_chart(fig, use_container_width=True)
```

#### 4.5.3 故障分布

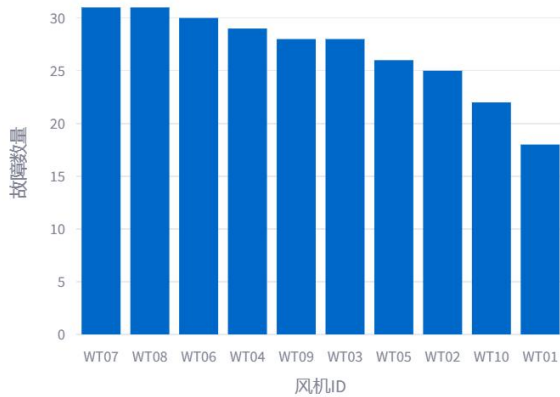
通过函数计算出计算所有风机的故障数量，生成柱状图展示每个风机的故障分布。按小时统计故障发生的时间分布，生成折线图展示故障随时间的变化趋势。具体界面如图 4.8 所示。

核心代码如下：

```
fig = px.bar(turbine_counts, x="风机 ID", y="故障数量", title="各风机故障数量")
st.plotly_chart(fig, use_container_width=True)
fig = px.line(hour_counts, x="小时", y="故障数量", title="故障时间分布")
fig.update_xaxes(tickmode='linear', dtick=1)
st.plotly_chart(fig, use_container_width=True)
```

## 故障分布

各风机故障数量



故障时间分布

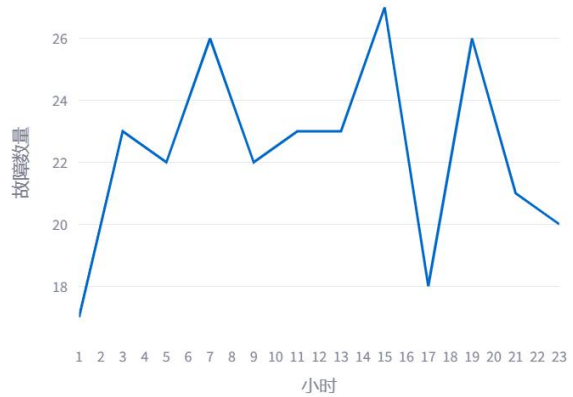


图 4.8 故障分布统计图

## 4.6 数据分析

用户选择风机和日期范围后，系统会筛选数据，将风速数据生成风速趋势图和风速分布直方图。将风向数据进行分析，生成风速与风向的极坐标散点图和风向玫瑰图。通过计算得到相关性矩阵，并以热力图形式展示变量间的相关性。最后对时间序列数据进行汇总分析，生成每日统计数据表，并提供导出功能。具体界面如图 4.9 所示。

### 风速数据分析

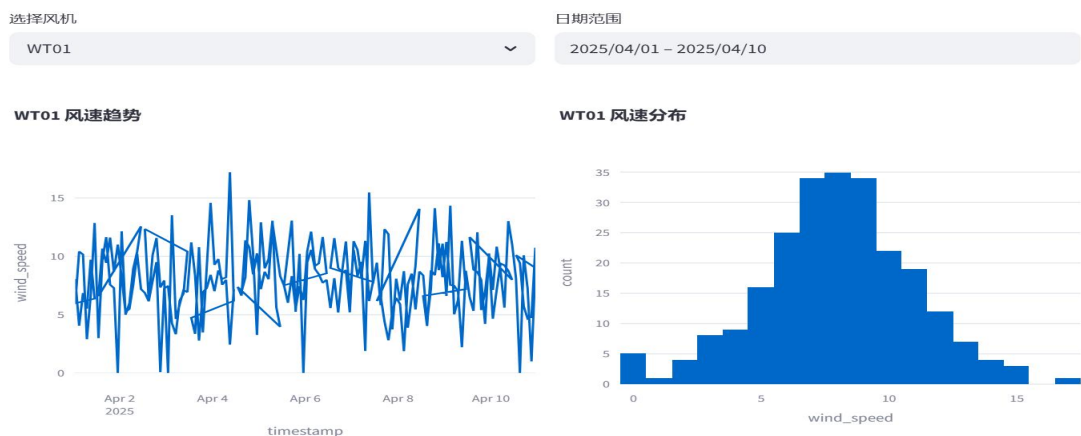


图 4.9 数据分析界面



核心代码如下：

```
fig = px.line(df_filtered, x= "timestamp" ,
              y= "wind_speed" ,
              title=f "{selected_turbine} 风速趋势" )
st.plotly_chart(fig, use_container_width=True)
fig = px.histogram(df_filtered, x= "wind_speed" , nbins=20,
                  title=f "{selected_turbine} 风速分布" )
st.plotly_chart(fig, use_container_width=True)
fig = px.scatter_polar(df_filtered, r= "wind_speed" ,
                      theta= "wind_direction" ,
                      title=f "{selected_turbine} 风速和风向分布" )
fig.update_layout(polar=dict(radialaxis=dict(showticklabels=True, ticks="")))
st.plotly_chart(fig, use_container_width=True)
fig = px.bar_polar(direction_counts, r= "频次" ,
                  theta= "方向" ,
                  title=f "{selected_turbine} 风向玫瑰图" )
st.plotly_chart(fig, use_container_width=True)
fig = px.imshow(correlation_matrix, x=correlation_matrix.columns,y=correlation_matrix.
columns,color_continuous_scale= "RdBu_r" ,title= "相关性矩阵" )
st.plotly_chart(fig, use_container_width=True)
df_daily = df_filtered.groupby(pd.Grouper(key="timestamp", freq="D")).agg({"wind_spe
ed": ["mean", "min", "max", "std"],"wind_direction": ["mean", "std"],"temperature": "mean",
"humidity": "mean","air_pressure": "mean"}).reset_index()
st.dataframe(df_daily, use_container_width=True)
```

## 4.7 故障分析

### 4.7.1 故障统计与分布

系统调用 `detect_wind_speed_faults` 和 `detect_wind_direction_faults` 函数分别检测风速和风向传感器的故障数据。将两部分故障数据合并为 `all_faults`。并显示总故障数、风速传感器故障数和风向传感器故障数。并绘制饼状图展示故障类型的分布情况。如果有足够数据，生成月度故障分布的柱状图；否则提示无数据。具体界面如图 4.10 所示。

核心代码如下：

```
st.metric(label= "总故障数" , value=len(all_faults), delta=None)
st.metric(label= "风速传感器故障" , value=len(wind_speed_faults), delta=None)
```



```

st.metric(label= “风向传感器故障”, value=len(wind_direction_faults), delta=None)
fig = px.pie(fault_counts, names= “故障类型”,
            values= “数量”,
            title= “故障类型分布”)
st.plotly_chart(fig, use_container_width=True)
fig = px.bar(monthly_counts, x= “月份”,
            y= “数量”,
            color= “故障类型”,
            title= “月度故障分布”)
st.plotly_chart(fig, use_container_width=True)

```

## 故障分布

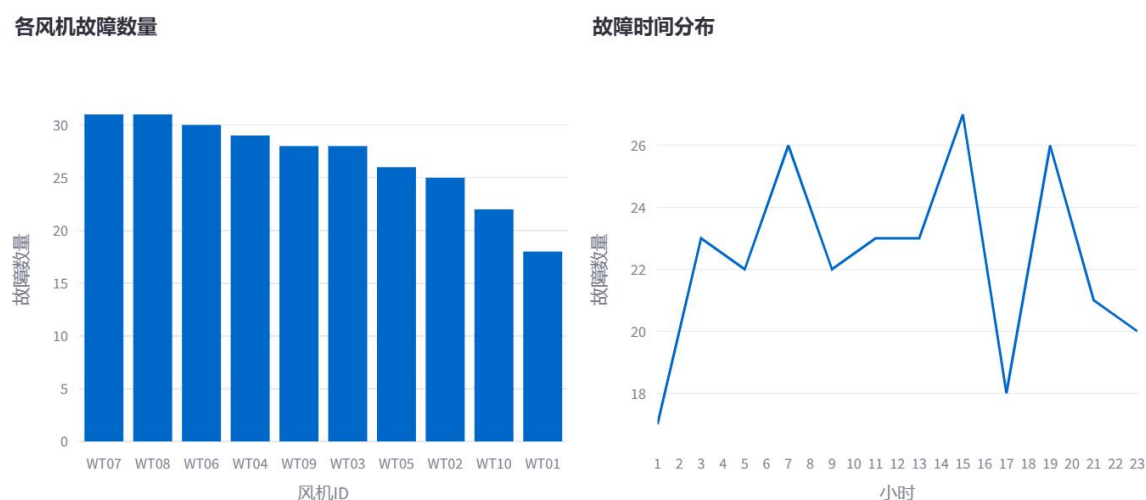


图 4.10 统计与分布界面

### 4.7.2 关联与预测

第一，系统调用 `detect_wind_speed_faults` 和 `detect_wind_direction_faults` 检测风速和风向的故障数据。并将两种故障数据合并生成 `all_faults` 数据框。如果存在故障数据，则按日期和风机 ID 生成故障计数矩阵，绘制相关性矩阵图。同时识别高度相关的风机对，给出检查建议。第二，用户通过选择风机和预测天数。模拟生成未来天数的故障概率数据。并将预测结果以折线图的方式展示出来，标记出高风险阈值（0.2）。并在下面提示高风险日期或无风险信息。具体界面如图 4.11 所示。

核心代码如下：

```

fig = px.imshow(fault_correlation,x=fault_correlation.columns,
                y=fault_correlation.columns,
                color_continuous_scale= “RdBu_r” ,

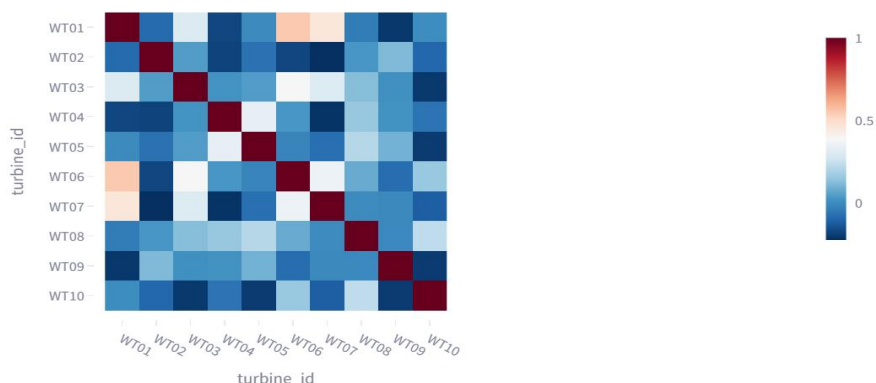
```

```

title= “风机故障相关性矩阵” )
st.plotly_chart(fig, use_container_width=True)
fig = px.line(forecast_df, x= “日期” ,
              y= “故障概率” ,
              markers=True,
              title=f “ {selected_turbine} 未来 {prediction_days} 天故障概率预测” )
fig.add_hline(y=0.2, line_dash= “dash” ,
              line_color= “red” ,
              annotation_text= “高风险阈值” )
st.plotly_chart(fig, use_container_width=True)

```

风机故障相关性矩阵



故障相关性分析结果

未发现高度相关的故障模式

## 故障预测模型

基于历史数据的故障预测

选择风机进行预测

WT01

预测未来天数



运行预测模型

图 4.11 关联与预测页面

## 4.8 参数设置

系统首先会检查用户角色是否为“管理员”或“工程师”，若不是则显示警告并退出。在故障检测参数设置模块，系统会通过 `get_db_connection` 方法连接数据库，查询当前故障检测参数阈值。并且将当前的阈值信息在界面上进行显示。用户根据具体情况对阈值信息进行设置。在传感器校准和数据采集设置模块中，系统会首先确定是否存在相应的数据表，如果不存在会进行相应的创建，并在界面上显示初始设置参数，用户也可以根据具体情况

的数值进行设置。具体界面如图 4.12 所示。

## 参数设置

### 故障检测参数设置

|                     |                                                  |   |   |          |        |   |   |
|---------------------|--------------------------------------------------|---|---|----------|--------|---|---|
| 风速下限 (m/s)          | 0.50                                             | - | + | 风向下限 (度) | 0.00   | - | + |
| 风速上限 (m/s)          | 25.00                                            | - | + | 风向上限 (度) | 359.90 | - | + |
| 数据变异阈值              | 0.020                                            | - | + | 连续相同值阈值  | 20     | - | + |
| 信号丢失阈值 (分钟)         | <div><div>1</div><div>5</div><div>30</div></div> |   |   |          |        |   |   |
| <div>保存故障检测参数</div> |                                                  |   |   |          |        |   |   |

### 传感器校准设置

|           |      |   |   |             |      |   |   |
|-----------|------|---|---|-------------|------|---|---|
| 风速校准系数    | 1.00 | - | + | 温度偏移量 (°C)  | 0.00 | - | + |
| 风向偏移量 (度) |      |   |   | 气压偏移量 (hPa) |      |   |   |

图 4.12 参数设置界面

核心代码如下（以故障检测参数设置为例）：

```
if st.button(“保存故障检测参数”):
    try:
        current_time = datetime.datetime.now().strftime(“%Y-%m-%d %H:%M:%S”)
        thresholds_data = [(“wind_speed_min”, wind_speed_min, current_time),
                             ( “wind_speed_max”, wind_speed_max, current_time),
                             ( “wind_direction_min”, wind_direction_min, current_time),
                             ( “wind_direction_max”, wind_direction_max, current_time),
                             ( “data_variance_threshold”, data_variance_threshold, current_time),
                             ( “consecutive_identical_threshold”, consecutive_identical_threshold, current_time),
                             ( “signal_loss_threshold”, signal_loss_threshold, current_time)]
        for name, value, timestamp in thresholds_data:
```

```
cursor.execute(
    "UPDATE fault_thresholds SET value = ?, last_updated = ? WHERE na
me = ?", (value, timestamp, name) )
#更新会话状态中的阈值
st.session_state.fault_thresholds = {"wind_speed_min": wind_speed_min,"
    wind_speed_max": wind_speed_max,
    "wind_direction_min": wind_direction_min,
    "wind_direction_max": wind_direction_max,
    "data_variance_threshold": data_variance_threshold,
    "consecutive_identical_threshold": consecutive_ident
        ical_threshold,
    "signal_loss_threshold": signal_loss_threshold}
st.success( “故障检测参数已成功保存！ ” )
```

4.9 用户管理

系统会先检查当前用户的角色是否为“管理员”，若不是则提示无权限并退出。权限通过后，系统会从数据库中获取所有用户信息，格式化后以表格形式进行展示。其次，管理员在界面可以注册不同角色的用户，并且可以对除管理员权限以外的用户修改信息，也可对用户进行删除操作。具体界面如图 4.13 所示。

用户管理

用户列表

|   | 用户名      | 姓名    | 角色  | 邮箱                   | 电话          | 上次登录                | 注册日期                |
|---|----------|-------|-----|----------------------|-------------|---------------------|---------------------|
| 0 | admin    | 系统管理员 | 管理员 | admin@huadian.com    | 13800138000 | 2025-04-25 21:07:09 | 2025-04-10 13:02:01 |
| 1 | engineer | 维护工程师 | 工程师 | engineer@huadian.com | 13900139000 | 2025-04-08 16:30:00 | 2025-04-10 13:02:01 |
| 2 | operator | 系统操作员 | 操作员 | operator@huadian.com | 13700137000 | 2025-04-09 09:15:00 | 2025-04-10 13:02:01 |
| 3 | wazhe    | s1    | 操作员 | 1950384890@qq.com    | 17713169056 | 尚未登录                | 2025-04-10 19:04:14 |
| 4 | wa3123   | s23   | 操作员 | 1950384890@qq.com    | 17713169056 | 尚未登录                | 2025-04-10 19:04:32 |
| 5 | 12       | hyt   | 管理员 | 1950384890@qq.com    | 15531329239 | 2025-04-15 09:13:10 | 2025-04-10 19:12:37 |
| 6 | 123      | hty   | 操作员 | 1950384890@qq.com    | 15531329239 | 2025-04-15 09:13:25 | 2025-04-11 08:42:24 |
| 7 | 233      | gh    | 工程师 | 1950384890@qq.com    | 15531329239 | 2025-04-15 09:12:48 | 2025-04-11 08:46:21 |
| 8 | phl      | 裴浩然   | 操作员 | 12891172489@qq.com   | 18567825431 | 尚未登录                | 2025-04-15 11:06:23 |

添加新用户

用户名

姓名

图 4.13 用户管理界面

核心代码如下：

```

for user in users:
    user_data.append({ "用户名": user['username'], "姓名": user['name'], "角色":
user['role'], "邮箱": user['email'], "电话": user['phone'], "上次登录": user['last_login'] or "尚未登录", "注册日期": user['registration_date']})
    user_df = pd.DataFrame(user_data)
    st.dataframe(user_df, use_container_width=True)
    #保存修改信息
    if reset_password and new_password:
        hashed_password = hashlib.sha256(new_password.encode()).hexdigest()
        cursor.execute("UPDATE users SET role = ?, name = ?, email = ?, phone = ?,
password = ? WHERE username = ?",(edit_role, edit_name, edit_email, edit_phone, hashed_password, user_to_edit))
    else:
        cursor.execute("UPDATE users SET role = ?, name = ?, email = ?, phone = ?
WHERE username = ?",(edit_role, edit_name, edit_email, edit_phone, user_to_edit))
    #删除用户
    if user_to_delete == st.session_state.username:
        st.error( "不能删除当前登录的用户。" )
    else:
        try:
            cursor.execute( "DELETE FROM users WHERE username = ?" , (user_to_delete,))

            conn.commit()
            st.success(f "用户 {user_to_delete} 已成功删除！" )
            st.rerun()
        except Exception as e:
            st.error(f "删除用户失败: {str(e)}" )

```

## 4.10 系统管理

系统会检查当前用户角色是否为“管理员”，若不是“管理员”，则显示警告信息并退出函数。在界面中首先会显示系统状态，包括系统运行时间、CPU 使用率和内存使用率。其次会显示数据库状态，包括数据库连接数和数据库大小。第三，会显示最近 10 条系统日志，包含时间戳、日志级别和信息。第四，会显示系统备份信息，包括上次备份时间和立即备份按钮功能。最后，提供自动备份设置和系统维护设置服务，并提供保存按钮。具体界面如图 4.14 所示。

# 系统管理

## 系统状态

系统运行时间

327天12小时

当前CPU使用率

23%

当前内存使用率

45%

## 数据库状态

数据库连接数

12

数据库大小

1.2 GB

↑ +0.1 GB

## 系统日志

|   | 时间戳                 | 日志级别    | 信息       |
|---|---------------------|---------|----------|
| 0 | 2025-04-25 23:02:23 | ERROR   | 系统日志信息 1 |
| 1 | 2025-04-25 22:02:23 | ERROR   | 系统日志信息 2 |
| 2 | 2025-04-25 21:02:23 | ERROR   | 系统日志信息 3 |
| 3 | 2025-04-25 20:02:23 | INFO    | 系统日志信息 4 |
| 4 | 2025-04-25 19:02:23 | INFO    | 系统日志信息 5 |
| 5 | 2025-04-25 18:02:23 | INFO    | 系统日志信息 6 |
| 6 | 2025-04-25 17:02:23 | INFO    | 系统日志信息 7 |
| 7 | 2025-04-25 16:02:23 | WARNING | 系统日志信息 8 |

图 4.14 系统管理界面

核心代码如下：

```
for i in range(10):
    date = (datetime.datetime.now() - datetime.timedelta(hours=i)).strftime("%Y-%m-%d %H:%M:%S")
    logs.append(
        { "时间戳": date, "日志级别": random.choice([ "INFO", "WARNING", "ERROR" ]), "信息": f"系统日志信息 {i + 1}" })
log_df = pd.DataFrame(logs)
st.dataframe(log_df, use_container_width=True)
if st.button( "立即备份" ):
    with st.spinner( "正在备份系统数据..." ):
        time.sleep(2)
    st.success( "系统备份完成！" )
```

### 4.11 数据库管理

此页面仅允许管理员角色访问页面。在此页面中统计并显示各表的数据量，同时允许用户指定路径进行数据库文件的备份。此外还提供了数据库优化和完整性检查功能。并且支持将表数据导出为 CSV 文件或从 CSV 文件导入数据到指定表。具体页面如图 4.15 所示。

# 数据库管理

## 数据库状态

|     |       |         |
|-----|-------|---------|
| 用户数 | 风电机组数 | 风速风向数据量 |
| 9   | 10    | 7,440   |

## 数据库备份

备份文件路径

backup/wind\_farm\_backup.db

立即备份数据库

## 数据库维护

优化数据库

检查数据库完整性

## 数据导入导出

选择要导出的表

users

导出数据

数据导入功能

此功能可用于从csv文件导入数据到数据库中

选择要导入的表

users

图 4.15 数据库管理界面

核心代码如下：

if st.button(“导出数据”):

try:

cursor.execute(f“SELECT \* FROM {table\_to\_export}”)

rows = cursor.fetchall()

data = [dict(row) for row in rows]

df = pd.DataFrame(data)

csv = df.to\_csv(index=False).encode('utf-8')

st.download\_button(label=“下载 CSV 文件”， data=csv,file\_name=f“{table\_to\_export}\_export.csv”， mime=“text/csv”， )

## 第5章 系统测试

### 5.1 系统测试的目的

开展系统测试会对系统设计中提到的功能进行测试，验证功能是否都能实现。是否存在问题，并将发现的问题进行分析解决，以保证系统能够符合需求。除了功能测试之外，还会对系统的非功能需求，如安全性、兼容性、性能等方面进行测试，以保证系统的可靠性与稳定性。

### 5.2 功能性测试

1. 对系统的用户管理模块进行测试：在登陆界面通过输入正确的用户名与密码验证能否正常登录到系统当中；在登录界面输入错误的用户名或者密码，验证系统是否会给出相应提示；在登录界面点击注册按钮验证是否能跳转到注册页面。在注册页面填写必要的信息验证是否能正常注册；在注册页面输入错误信息和空出必填信息验证系统是否给出相应提示；在注册页面点击返回登录界面按钮查看是否能跳转回登陆界面。以管理员身份登录加入系统，进入用户管理界面，查看是否能够显示全部用户列表；点击下方的添加用户，输入相关信息，查看是否可以添加用户；编辑用户，查看保存是否正常；勾选用户，执行删除操作，查看是否删除并且用户列表不显示。

2. 对系统管理模块测试：查看系统运行时间、CPU、内存占用率，数据是否正确；查看数据库连接数、数据库大小，数据是否正确；查看日志，日志是否正确显示最近的日志信息；点击“立即备份”按键，系统是否备份出文件；设置自动备份参数，系统是否按照设置的频率进行系统备份。

3. 对测试故障数据模块：在故障数据页面，通过选择不同的风机、故障类型以及日期范围，测试是否能正确显示对应时间段内的故障数据；在相应的故障数据表格下方的导出按钮，测试输出的文件中的数据是否与表格数据相同。在数据分析界面，选择不同的风机与日期查看是否会生成正确的风速趋势图、风速分布图等图表；查看相关性矩阵，验证是否能正确显示各变量之间的相关性；点击导出分析数据查看导出数据是否一致。

4. 对参数设置模块进行测试：修改风速上下限、连续相同值阈值等参数，验证是否能成功保存并更新阈值；修改风速校准系数、风向偏移量、采样频率、平均周期、存储周期等参数等参数，查看是否能成功保存至数据库中。

5. 对数据库管理模块进行测试：在数据库管理界面查看备份路径并点击“立即备份数据库”按钮，查看是否生成备份文件；点击“优化数据库”按钮，查看是否能成功执行；点击“检查数据库完整性”，查看是否能正确返回检查结果；选择一个数据表并点击“导出数据”



按钮，查看是否能成功生成文件。上传一个 CSV 文件并点击“导入数据”，验证是否能成功将数据导入至指定的数据表中。

## 5.3 非功能性测试

### 5.3.1 安全性测试

1. 对用户的权限进行测试，使用不同角色（管理员、工程师、操作员）登录系统查看是否每个用户只能访问其权限范围内的界面内容。
2. 对数据的加密进行测试：打开数据库表查看 `users` 表中的密码是否以加密的形式存储在表中的。

### 5.3.2 可用性测试

1. 查看页面整洁和友好性，邀请同学对系统进行体验，他们是否觉得页面的整洁直观易用。
2. 验证系统的错误提醒功能，验证系统在发生错误时是否能清晰的显示错误信息。

## 第6章 结论与展望

### 6.1 结论

本研究针对于华电长青风电场测风系统开展了故障检测与诊断系统的设计。通过对测风系统组成结构、典型故障的分析，了解故障检测、诊断的重点和难点内容。

在故障检测方面，将多种算法在综合对比后进行选择，从而使得华电长青风电场测风系统能够更好地检测，提高故障检测的及时性和准确性，利用大量历史数据和实时运行数据，确保系统能够准确检测常见的故障问题（风速传感器故障、风向传感器故障、数据传输线路故障等问题），能够及时发现异常，为进一步诊断提供基础。

在故障诊断过程中根据传感器传来的不同的数据信息，给出不同的故障类型、故障建议，节省了故障诊断的时间，提高了故障处理效率。

在数据导入系统检测环节，通过模拟华青风电场相应的数据，可以将测风系统的故障全部检测出来。同时，该系统的应用可以降低风电场的运维成本，提高运维效率，具有显著的经济效益和实际应用价值。

### 6.2 未来展望

尽管本研究取得了一些成果，但是华电长青风电场的测风系统故障检测与诊断研究还有许多方面值得进一步的研究与完善。

在系统的性能方面，当前的故障检测和故障诊断模型虽然取得较好的成果，但在更复杂的情况例如天气条件相对恶劣、测风系统设备运行老化程度较高时，故障诊断的精度有待提升，后续可以考虑引入更多的先进算法，如生成对抗网络（GAN）等，丰富模型，强化模型对复杂故障模式的学习和识别能力。

针对故障诊断，可建立深度学习诊断模型，发挥卷积神经网络 CNN 和循环神经网络 RNN 的优势，对故障特征深度学习进行故障分类诊断，不断优化数据加工流水线，优化数据处理流程，提高数据品质，提升系统性能。

拓展故障类型的覆盖范围方面，目前研究主要集中在常见故障，而对少数发生且严重影响到测风系统运行的故障类型还未涉及，在今后可以搜集更多风电场测风系统以前发生的历史数据，如对少数发生的故障类型，建立更多的故障样本库，将故障类型进行更多的分类，研究更深层次的故障类型，完善故障检测诊断系统对不同故障的诊断能力。

## 参考文献

- [1] 周安然. 测风激光雷达控制系统的研制与风场观测[D]. 导师: 孙东松. 中国科学技术大学, 2020.
- [2] 杜宗印,卿珊珊,陆虎.创新大赛引领下大学生创新实践教育研究——以基于海上测风塔的风电环境监测系统设计为例[J].江苏科技信息,2025,42(02):41-46.
- [3] 潘向阳. 复杂噪声背景下的超声波测风方法研究[D]. 导师: 石屹然. 吉林大学, 2023.
- [4] 杨应鹏. 基于改进时差法的超声波测风系统研究[D]. 导师: 陈晓. 南京信息工程大学, 2020.
- [5] 袁逸博, 何熠, 朱碧泓, 顾晨, 宋励奋, 陈冬冬, 裘建宇, 杨闯. 基于连续波激光雷达的漂浮式测风系统的测量数据分析验证[J]. 应用海洋学学报.
- [6] 刘宣廷. 基于数字图像相似度建模的激光测风系统研究[D]. 导师: 王中秋. 齐鲁工业大学, 2024.
- [7] 商志伟. 极低温度环境下的超声波测风传感器测控系统研制[D]. 导师: 漆随平. 齐鲁工业大学, 2023.
- [8] 薛昂, 贾豫东, 李琛. 相干测风激光雷达功率谱估计硬件系统综述[J]. 科技与创新, 2024, (01): 115-117.
- [9] 郝玉珠,石岚,贾晓红.风电机组异常风速的识别和修正方法研究[J].气象科技,2024,52(05):644-651.
- [10] 王乐.风向风速传感器的原理及其检测研究[J].中国设备工程,2022,(20):166-168.
- [11] 徐骁.区域站风向风速故障判断与维修[J].电脑知识与技术,2020,16(19):232-233.
- [12] 焦杰,文泽军.基于模糊灰关联分析法的直驱式风电机组故障树分析[J].湖南科技大学学报(自然科学版),2020,35(01):62-70.
- [13] 袁红亮,陈彬, 刘玮,宋俊博,王炎.激光雷达测风的可靠性验证[J].西北水电,2022,(04):103-107.
- [14] 孙丹丹.测风传感器故障检测仪的设计与应用[J].无线互联科技,2022,19(03):77-78.
- [15] 焦杰.大型直驱式风电机组故障分析与维护优化[D].湖南科技大学,2020.
- [16] 彭坚,王小兰,马丽.不同测风仪外场试验与分析研究[J].实验室检测,2025,3(04):81-84.
- [17] 李浩琪.模块化手持自动气象站测风模型研究及系统设计[D].南京信息工程大学,2024.DOI:10.27248/d.cnki.gnjqc.2024.001487.
- [18] 闫宇琛.全光纤非相干多普勒激光雷达系统设计及实验[D].西安理工大学,2024.DOI:10.27398/d.cnki.gxalu.2024.000122.
- [19] 张思妍.全光纤非相干多普勒测风激光雷达鉴频系统对比分析及频率校正方法[D].西安理工大学,2024.DOI:10.27398/d.cnki.gxalu.2024.001810.
- [20] 张俭.基于深度学习的风功率预测模型的研究[D].北京交通大学,2019.

## 致谢

行文至此，落笔为终。完成此篇论文意味着我的大学旅程即将结束，心中满是感恩与不舍。

首先要向我的导师董颖霞教授致以最崇高的敬意与最诚挚的感谢。从选题的反复斟酌，到软件设计的精心指导，再到论文初稿无数次的细致批注，导师始终以渊博的学识、严谨的治学态度为我指明方向。记得在数据采集遇到瓶颈时，导师将风电场工作人员的联系方式告诉了我，并与其进行沟通，那份负责与耐心，不仅化解了我的焦虑，更让我深刻领悟到学术探索的真谛。

其次感谢同学们在软件设计过程中给予的无私帮助，在软件代码频繁报错与思路中断时，是郭旭同学帮我理清思路解决问题。在我陷入思维困境时，他们用温暖的鼓励和轻松的玩笑，帮我驱散阴霾，重拾信心。

最后，我要将最深的爱与感激献给我的家人。父母多年来默默的支持与付出，是我前行的坚实后盾；他们在我迷茫时的谆谆教诲，给予我直面困难的勇气。没有他们，这篇论文的完成将无从谈起。

以上这些恩情，我将铭记于心，并以此为动力，在未来的道路上继续奋勇前行。